

The LangChain Practitioner's Guide

A complete, hands-on path to building LLM applications — explained from the ground up, with every example shown twice: once for **Ollama** (local) and once for the **Gemini API** (cloud).

THIS VOLUME COVERS

Phase 0 — Prerequisites

Phase 1 — LangChain Core

How to read this guide

This series takes you from an empty terminal to a deployed, traceable LLM application. It is split into six parts that mirror the six learning phases. You are holding **Part 1**, which combines the short setup phase (Phase 0) with your first real LangChain code (Phase 1).

Three promises shape every page:

- **Self-contained.** When you finish a part, you should not need to open another website to understand what you just read. Concepts are defined where they first appear.
- **Two engines, always.** Every runnable example appears twice — once using **Ollama**, which runs models on your own machine for free, and once using the **Gemini API**, Google's hosted models. The LangChain code around them is almost identical, which is exactly the point of the framework.
- **Accurate for today.** LangChain changed substantially at its 1.0 release. Older tutorials you may find online use classes that have since moved to a separate `langchain-classic` package. Everything here uses the current 1.x interface.

Conventions used in this book

Terminal blocks are commands you type into a shell. **Python** blocks are code you place in a `.py` file or notebook. **Output** blocks show what you should expect to see.

Lines beginning with `#` in Python (or `#` after a command) are comments — explanations for you, ignored by the computer.

The six-phase roadmap

0	Prerequisites venv, .env keys, Python dicts, Ollama & Gemini setup	THIS PART
1	LangChain Core Chat models, invoke vs stream, PromptTemplate	THIS PART
2	Chains & LCEL The pipe operator, memory, a multi-turn chatbot	PART 2
3	RAG & Embeddings Load, chunk, embed, retrieve — “chat with a PDF”	PART 3
4	Agents & Tools @tool, ReAct agents, a taste of LangGraph	PART 4
5	Production LangSmith tracing, streaming, a FastAPI endpoint	PART 5

Prerequisites

You likely have most of this already. The goal of this phase is to make sure four small habits are second nature before any LangChain code runs: isolating projects with **virtual environments**, keeping secrets in **.env files**, working comfortably with **Python dictionaries**, and having both of our model engines — **Ollama** and the **Gemini API** — installed and reachable.

0.1 Virtual environments (`venv`)

A virtual environment is a private, throwaway copy of Python that belongs to *one* project. Anything you install while it is active stays inside that project's folder instead of polluting the Python that ships with your operating system. This matters because LangChain projects pull in many packages, and two projects often need different versions of the same one. A venv keeps them from fighting.

Create and activate one inside your project folder:

```

# 1. Move into (or create) your project folder
mkdir langchain-course && cd langchain-course

# 2. Create a virtual environment named ".venv"
python3 -m venv .venv

# 3. Activate it --- macOS / Linux
source .venv/bin/activate

# 3. Activate it --- Windows (PowerShell)
.venv\Scripts\Activate.ps1
```

Once active, your shell prompt usually shows `(.venv)` at the start of the line. That is your signal that `python` and `pip` now point at the isolated copy. Install a package to confirm everything works:

```

pip install --upgrade pip      # get the latest installer
pip install python-dotenv      # we will use this in the next section
pip list                      # see what is installed in THIS environment
```

When you are done working, leave the environment with a single command:

```
deactivate
```

+ Always activate before you work

If you open a new terminal tomorrow, the environment is *not* active automatically — you must run the `activate` command again from the project folder. Forgetting this is the single most common reason a beginner sees “ModuleNotFoundError: No module named langchain” even though they “already installed it.”

i Keep .venv out of version control

If you use Git, add a file named `.gitignore` containing the line `.venv/` so the (large, machine-specific) environment is never committed. You recreate it on another machine from your `requirements.txt`.

0.2 Secrets in `.env` files with `python-dotenv`

Your Gemini API key is a password. If you paste it directly into your Python files, it will eventually end up on GitHub, in a screenshot, or in a shared notebook — and a leaked key can cost you real money. The standard fix is to keep secrets in a plain text file called `.env` that lives next to your code but is **never** committed, and to load it at runtime with the `python-dotenv` package.

Create a file named exactly `.env` in your project root:

```
.ENV

# .env -- no spaces around "=", no quotes needed
GOOGLE_API_KEY=AIzaSyXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

# Ollama runs locally and needs no key, but you can point at a
# non-default host if you ever run it on another machine:
OLLAMA_HOST=http://localhost:11434
```

Load those values at the very top of your program, before any LangChain import that needs them:

```
PYTHON

import os
from dotenv import load_dotenv

load_dotenv()  # reads .env and puts values into the environment

api_key = os.getenv("GOOGLE_API_KEY")  # fetch a value; returns None if missing
print("Key loaded:", bool(api_key))    # never print the key itself!
```

! Tell Git to ignore it

Add `.env` to your `.gitignore` immediately. A good starter `.gitignore` for these projects is just three lines: `.venv/`, `.env`, and `__pycache__/`.

Why this works so smoothly with LangChain: the provider integrations look for their keys in the environment by default. Once `load_dotenv()` has run, `ChatGoogleGenerativeAI` will find `GOOGLE_API_KEY` on its own — you usually do not pass the key in code at all.

0.3 Python dictionaries — the currency of LangChain

A dictionary (`dict`) maps **keys** to **values**. LangChain leans on dicts heavily: when you fill in a prompt template, you hand it a dict; when a chain runs, data flows from step to step as dicts. Being fluent here removes most of the confusion beginners feel about “where the variables go.”

```
PYTHON

# Create a dict: keys on the left of ":", values on the right
user = {"name": "Ada", "topic": "vectors", "level": "beginner"}

# Read a value by its key
print(user["topic"])          # -> vectors

# Safe read with a fallback if the key might be missing
print(user.get("language", "English")) # -> English (key absent, default used)

# Add or update a key
user["level"] = "intermediate"

# Loop over keys and values together
for key, value in user.items():
    print(key, "=", value)
```

LangChain prompt templates expect exactly this shape. A template with placeholders `{topic}` and `{level}` is “filled” by passing a dict whose keys match those names:

```
PYTHON

# The dict keys MUST match the placeholder names in the template
inputs = {"topic": "vectors", "level": "beginner"}

# (we will write the real template in Phase 1; this is the shape it consumes)
# prompt.invoke(inputs) -> builds the final text from the dict
```

Nested dicts show up constantly

Configuration and message payloads are often dicts inside dicts, e.g. `{"configurable": {"thread_id": "abc"}}`. Reach inside with chained keys: `config["configurable"]["thread_id"]`. You will meet this exact pattern when we add memory in Part 2.

0.4 Installing Ollama (your free local engine)

Ollama downloads open-weight models and runs them on your own computer. There is no API key and no per-token cost, which makes it perfect for learning and for privacy-sensitive work. The trade-off is that larger models need a capable machine.

1. Install the Ollama application. Download it for macOS, Windows, or Linux from ollama.com. On macOS you can also use Homebrew:

```
TERMINAL  
  
brew install ollama      # macOS via Homebrew (optional)  
brew services start ollama # start it as a background service
```

2. Make sure the server is running and pull a model. Ollama exposes a local server on port [11434](#). Pull a small, capable general model and an embedding model (we will need the embedding one in Part 3):

```
TERMINAL  
  
ollama serve      # starts the local server (skip if installed as a service)  
  
ollama pull llama3.1      # a solid 8B general chat model  
ollama pull nomic-embed-text # an embedding model for later (RAG)  
  
ollama list          # show everything you have downloaded
```

3. Sanity-check it straight from the terminal before involving Python:

```
TERMINAL  
  
ollama run llama3.1 "Say hello in one short sentence."
```

+ Sizing rule of thumb

An “8B” (8-billion parameter) model such as [llama3.1](#) wants roughly 8 GB of free RAM. If your machine is modest, start with a smaller tag like [llama3.2:1b](#). The first request after startup is slow because the model is being loaded into memory; subsequent ones are fast.

0.5 Getting a Gemini API key (your cloud engine)

Google’s Gemini models run in the cloud and are reached with an API key. The fastest way to get one is **Google AI Studio** (aistudio.google.com): sign in, open the “Get API key” screen, create a key, and copy it.

Put the key into the `.env` file you made in section 0.2:

```
.ENV
GOOGLE_API_KEY=AIzaSyXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

That is the whole setup. When we initialise a Gemini chat model in Phase 1, the LangChain integration reads `GOOGLE_API_KEY` from the environment automatically after `load_dotenv()` has run.

WHICH GEMINI MODEL NAME?

Model names evolve quickly. As of 2026 the dependable, low-cost default is `gemini-2.5-flash`; newer, faster options such as `gemini-3-flash` and the high-reasoning `gemini-3-pro` are also available through the same API. This guide uses `gemini-2.5-flash` in examples because it is broadly available and inexpensive — swap the string for any model your account can access.

OLLAMA VS GEMINI AT A GLANCE

	Ollama	Gemini API
Runs	On your machine	In Google’s cloud
Cost	Free	Per token
Key needed	No	Yes
Best for	Learning, privacy	Top quality, scale

★ You are ready for Phase 1

A checklist: an activated `.venv`, a `.env` holding `GOOGLE_API_KEY`, Ollama installed with `llama3.1` pulled, and comfort reading a Python dict. With those in place, the rest of this part is pure LangChain.

PHASE 1

LangChain Core

This is your “hello world” moment. By the end of this phase you will install LangChain, talk to a model with a single line of code, understand the crucial difference between **invoke()** and **stream()**, and master the **PromptTemplate** — the single most-used building block in the whole framework. Every example is shown for both Ollama and Gemini.

1.1 What LangChain actually is

LangChain is an open-source framework for building applications powered by large language models. Its core promise is a **single, standard interface** over many different model providers. You write your application logic once; switching from a local Ollama model to cloud Gemini (or to OpenAI, Anthropic, and others) becomes a one-line change rather than a rewrite of your networking code.

In October 2025 LangChain reached its **1.0** release. Two facts from that release matter for everything you will read:

- The main `langchain` package was deliberately slimmed down to the essentials. Older helper classes (such as the legacy `LLMChain`) were moved into a separate package called `langchain-classic`. If a tutorial imports from there, it is using the pre-1.0 style.
- The framework is organised in layers. `langchain-core` defines the universal interfaces (chat models, messages, prompts, output parsers). Thin **provider packages** — like `langchain-ollama` and `langchain-google-genai` — implement those interfaces for a specific backend. High-level **agents** are built on `langgraph`, which we reach in Part 4.

Why the layering helps you

Because every provider implements the same `langchain-core` interface, the prompts, parsers, and chains you learn now work unchanged whether the model runs locally or in the cloud. That is the entire reason this guide can show one set of concepts and merely swap the engine.

1.2 Installation

With your virtual environment active (Phase 0.1), install the core framework plus the two provider packages this series uses:

```

● ● ●  TERMINAL

pip install -U langchain           # the framework
pip install -U langchain-ollama    # local models via Ollama
pip install -U langchain-google-genai # Gemini models via Google
pip install -U python-dotenv       # load the .env file (from Phase 0)
```

Confirm the install and check the version you are on:

```

● ● ●  TERMINAL

python -c "import langchain; print(langchain.__version__)"
# Expect a 1.x version, e.g. 1.3.2
```

+ One framework, many engines

You only ever install `langchain` once. Each **engine** you want to use gets its own small package (`langchain-ollama` , `langchain-google-genai` , and so on). Installing a provider package does not change any code you have already written — it simply makes that backend available.

1.3 Chat models: the core object

Modern LLMs are **chat models**. Instead of a plain “text in, text out” function, they take a list of **messages** (each with a role — system, human, or assistant) and return a single assistant message. LangChain gives every chat model the same methods, the most important being `invoke()`, `stream()`, and `batch()`.

Let’s create a chat model with each engine. First the local one:

```
PYTHON (OLLAMA)

from langchain_ollama import ChatOllama

# Talks to the Ollama server you started in Phase 0. No API key.
llm = ChatOllama(model="llama3.1", temperature=0)

response = llm.invoke("Say hello in one short sentence.")
print(response.content)
```

Now the cloud one. Notice how little changes — only the import and the class:

```
PYTHON (GEMINI)

from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI

load_dotenv() # makes GOOGLE_API_KEY available

llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)

response = llm.invoke("Say hello in one short sentence.")
print(response.content)
```

The `temperature` parameter controls randomness: `0` makes answers focused and repeatable, higher values (up to `~1`) make them more creative. The return value is an `AIMessage` object; its text lives in `.content`.

The unified initialiser: `init_chat_model`

Because swapping engines is so common, LangChain ships a single factory that builds the right chat model from a string. This is the most convenient way to keep one codebase that can run on either engine:



PYTHON

```
from dotenv import load_dotenv
from langchain.chat_models import init_chat_model

load_dotenv()

# Pick ONE of these lines. The format is "provider:model".
llm = init_chat_model("ollama:llama3.1", temperature=0) # local
# llm = init_chat_model("google_genai:gemini-2.5-flash", temperature=0) # cloud

print(llm.invoke("Name three primary colors.").content)
```



This is the toggle we will reuse

Throughout the series, the phrase “choose your engine” means: comment one `init_chat_model(...)` line and uncomment the other. The rest of the program does not change — that is the standard interface doing its job.

1.4 `invoke()` vs `stream()` (and `batch()`)

These two methods are how you actually run a model, and understanding the difference is essential.

`invoke()` sends your input and waits for the *entire* answer to come back as one object. `stream()` returns the answer in small pieces (“chunks”) as the model produces them — this is what powers the type-as-it-thinks effect you see in chat apps.

INVOKE — WAIT FOR THE WHOLE ANSWER

```
PYTHON

# Works identically for ChatOllama or ChatGoogleGenerativeAI
result = llm.invoke("Explain what an API is, in two sentences.")

print(type(result))      # <class 'langchain_core.messages.ai.AIMessage'>
print(result.content)    # the full text, available only once it is all ready
```

Use `invoke()` whenever you need the complete result before doing anything with it — for example when the next step must parse the full answer.

STREAM — PRINT PIECES AS THEY ARRIVE

```
PYTHON

# Loop over chunks; each chunk is a small AIMessageChunk
for chunk in llm.stream("Write a four-line poem about the sea."):
    print(chunk.content, end="", flush=True)  # end="" keeps it on one line
print()  # final newline once the stream is done
```

Streaming does not make the model faster overall, but it makes your application *feel* fast because the user sees words appear immediately instead of staring at a spinner. Both engines support it: Ollama streams tokens from your local server, Gemini streams them from Google's.

BATCH — MANY INPUTS AT ONCE

```
PYTHON

questions = [
    "Capital of France?",
    "Capital of Japan?",
    "Capital of Kenya?",
]
answers = llm.batch(questions)  # returns a list of AIMessage, in order
for q, a in zip(questions, answers):
    print(q, "->", a.content)
```

invoke or stream — how to choose

Reach for **stream()** when a human is watching the output appear (chatbots, live UIs). Reach for **invoke()** for backend steps, when you must validate or transform the whole answer, or inside larger chains. Use **batch()** to process a list of independent inputs efficiently.

1.5 Messages and roles

So far we passed a plain string, which LangChain treats as a single human message. For real control you pass a **list of messages**, each with a role. The two you use most are the **system** message (standing instructions that shape the model's behaviour) and the **human** message (the user's actual input). The model replies with an **AI** message.

```
PYTHON

from langchain_core.messages import SystemMessage, HumanMessage

messages = [
    SystemMessage("You are a terse assistant. Answer in at most five words."),
    HumanMessage("What is the capital of Australia?"),
]

result = llm.invoke(messages)
print(result.content)      # e.g. "Canberra."
```

This is identical for both engines — messages are a `langchain-core` concept, so the same list works whether `llm` is a `ChatOllama` or a `ChatGoogleGenerativeAI`. A convenient shorthand is a list of `(role, text)` tuples:

```
PYTHON

messages = [
    ("system", "You are a terse assistant. Answer in at most five words."),
    ("human", "What is the capital of Australia?"),
]

print(llm.invoke(messages).content)
```

Why the system message is powerful

The system message is where you set tone, persona, output format, and rules (“always answer in JSON”, “you are a Python tutor”). Because it is sent on every call, it steers the whole conversation. We will template it next so you can reuse it cleanly.

1.6 PromptTemplate — the most-used building block

Hard-coding prompts as f-strings scattered through your code becomes a maintenance nightmare. A **prompt template** treats a prompt like a reusable function: it has named placeholders, and you fill them by passing a dict (exactly the skill from Phase 0.3). LangChain offers two flavours.

PromptTemplate (a single block of text)

```
PYTHON

from langchain_core.prompts import PromptTemplate

template = PromptTemplate.from_template(
    "Explain {topic} to a {level} in exactly two sentences."
)

# Fill it by passing a dict whose keys match the placeholders
prompt_value = template.invoke({"topic": "embeddings", "level": "beginner"})
print(prompt_value.to_string())
# -> Explain embeddings to a beginner in exactly two sentences.
```

ChatPromptTemplate (a list of role-tagged messages)

For chat models you usually want a `ChatPromptTemplate`, which templates a whole list of messages — a system message plus a human message is the classic shape:

```
PYTHON

from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a patient tutor for {audience} learners."),
    ("human", "Explain {topic} in two sentences."),
])

messages = prompt.invoke({"audience": "beginner", "topic": "vector databases"})
for m in messages.to_messages():
    print(m.type, "->", m.content)
```

Now feed the filled template straight into either engine — the template output is exactly what a chat model expects:

```
PYTHON (OLLAMA)

# Ollama
from langchain_ollama import ChatOllama
llm = ChatOllama(model="llama3.1", temperature=0)
print(llm.invoke(prompt.invoke({"audience": "beginner", "topic": "APIs"})).content)
```

PYTHON (GEMINI)

```
# Gemini
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI
load_dotenv()
llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)
print(llm.invoke(prompt.invoke({"audience": "beginner", "topic": "APIs"})).content)
```

Partial templates

Sometimes one placeholder is fixed for a whole session. `.partial()` locks it in so callers only supply what changes:

PYTHON

```
base = ChatPromptTemplate.from_messages([
    ("system", "You teach {audience} learners."),
    ("human", "Explain {topic}."),
])

beginner_prompt = base.partial(audience="beginner") # audience now fixed
msgs = beginner_prompt.invoke({"topic": "tokens"}) # supply only "topic"
```

1.7 Connecting the pieces (a first taste of the pipe)

You now have the three core objects: a **prompt template**, a **chat model**, and a way to read the result. LangChain lets you connect them with the `|` ("pipe") operator, read left to right like a production line: the prompt's output flows into the model. Adding a **StrOutputParser** at the end converts the `AIMessage` into a plain string, so the whole pipeline returns clean text.

```
PYTHON

from dotenv import load_dotenv
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()

# --- choose your engine -----
llm = init_chat_model("ollama:llama3.1", temperature=0)
# llm = init_chat_model("google_genai:gemini-2.5-flash", temperature=0)
# -----

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a concise {role}.",),
    ("human", "{question}"),
])

chain = prompt | llm | StrOutputParser() # the pipeline

answer = chain.invoke({"role": "math tutor", "question": "Why is 0.1 + 0.2 not 0.3?"})
print(answer) # a plain string, ready to use
```

That single `chain` object also supports `stream()` and `batch()`, just like a bare model — everything composes. This pipe-based style is called **LCEL** (LangChain Expression Language), and it is the entire subject of Part 2. For now, simply notice that you built a real, reusable, engine-agnostic LLM pipeline in about ten lines.

i What StrOutputParser does

A chat model returns an `AIMessage`. `StrOutputParser()` is the simplest **output parser**: it just pulls out `.content` as a string. Later you will use parsers that turn model output into JSON or validated objects, but this one keeps our first pipeline tidy.

Build a calculator chatbot

Time to consolidate everything: a small command-line **calculator chatbot**. It keeps running, takes a math question on each line, and uses your chosen engine to answer with a short explanation. It exercises the whole of Phase 1 — environment loading, a chat model, a system prompt via a template, the pipe, an output parser, and streaming for a live feel — while staying entirely within “core” (no tools or memory yet; those arrive in later parts).

```
PYTHON (CALCULATOR_CHATBOT.PY)

# calculator_chatbot.py
from dotenv import load_dotenv
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()

# --- choose your engine -----
llm = init_chat_model("ollama:llama3.1", temperature=0)
# llm = init_chat_model("google_genai:gemin-2.5-flash", temperature=0)
# -----

prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are a careful calculator assistant. The user gives a math "
     "question. Work it out step by step, then end with a final line "
     "'Answer: <result>'. If the input is not a math question, say so politely."),
    ("human", "{question}"),
])

chain = prompt | llm | StrOutputParser()

print("Calculator chatbot. Type a math question, or 'quit' to exit.\n")
while True:
    question = input("you > ").strip()
    if question.lower() in {"quit", "exit"}:
        print("Goodbye!")
        break
    if not question:
        continue

    print("bot > ", end="", flush=True)
    for chunk in chain.stream({"question": question}): # live, streamed answer
        print(chunk, end="", flush=True)
    print("\n")
```

Run it from your activated environment:



TERMINAL

```
python calculator_chatbot.py
```

A sample session looks like this:



OUTPUT

```
Calculator chatbot. Type a math question, or 'quit' to exit.
```

```
you > what is 15% of 240?
```

```
bot > 15% means  $15/100 = 0.15$ .
```

```
 $0.15 \times 240 = 36$ .
```

```
Answer: 36
```

```
you > quit
```

```
Goodbye!
```



Make it your own

Try these small extensions to cement the concepts: (1) flip the engine line and confirm the behaviour is nearly identical; (2) change the system prompt to demand answers in JSON like

`{"steps": ..., "answer": ...}`; (3) lower or raise `temperature` and watch how the explanations change. Each tweak touches exactly one Phase 1 idea.

Part 1 recap & what comes next

You went from an empty folder to a working, engine-agnostic chatbot. Here is the ground you now stand on.

Phase 0 — you can now

- Create and activate a `venv` per project.
- Store secrets in `.env` and load them with `python-dotenv`.
- Read and fill Python dicts — LangChain's data currency.
- Run models locally with Ollama and reach Gemini with an API key.

Phase 1 — you can now

- Install LangChain 1.x and provider packages.
- Create a chat model two ways, including `init_chat_model`.
- Use `invoke`, `stream`, and `batch` correctly.
- Build prompts with `ChatPromptTemplate` and pipe a real chain.

★ Coming in Part 2 — Chains & LCEL

We go deep on the pipe operator you just met: branching and parallel steps, passing data between stages, **adding memory** so the bot remembers earlier turns, and an introduction to document loaders. The project is a genuine **multi-turn assistant** that holds a conversation — again, shown for both Ollama and Gemini.

End of Part 1 · The LangChain Practitioner's Guide